

Resolución del Práctico 1

Testing de Software 2024

Índice

Introducción	3
1. Ejercicio 1	3
2. Ejercicio 2	3
2.1. Diferencia entre <i>fault</i> y <i>failure</i>	3
3. Ejercicio 3	4
3.1. Programa 1: <code>findLast(int[] x, int y)</code>	4
3.2. Programa 2: <code>lastZero(int[] x)</code>	4
3.3. Programa 3: <code>countPositive(int[] x)</code>	5
3.4. Programa 4: <code>oddOrPos(int[] x)</code>	6
3.5. Síntesis del ejercicio	6
4. Ejercicio 4	7
4.1. Reparaciones implementadas	7
4.2. Casos de prueba JUnit	7
4.3. Detección de falla usando el modelo RIPR	7
4.4. Ejecución de pruebas	9
5. Ejercicio 5	9
5.1. Análisis de <code>Point</code>	9
5.2. Análisis de <code>ColorPoint</code>	10
Ejercicio 6	10
6.1. Aclaración sobre el material provisto	10
6.2. a) Nuevos tests, falla detectada y corrección	11
6.3. b) Identificación de Arrange–Act–Assert	11

6.4. Ejecución de pruebas	11
7. Ejercicio 7	12
7.1. Aclaración sobre el material provisto	12
7.2. Tests implementados	12
7.3. Resultado y análisis	12
7.4. Identificación de Arrange–Act–Assert	12
Conclusiones	13

Introducción

Este documento reúne la resolución del *Práctico 1* de *Testing de Software 2024*. La consigna combina lecturas del libro de Ammann y Offutt con ejercicios de análisis de defectos, reparación de programas y diseño de pruebas unitarias en JUnit. A lo largo del práctico se trabajan los conceptos de *fault*, *error* y *failure*, el modelo RIPR como herramienta para razonar sobre la detectabilidad de un defecto, y un caso clásico del libro de Bloch sobre el contrato de `equals` en jerarquías por herencia.

1. Ejercicio 1

Lectura de los capítulos 1 y 2 del libro *Introduction to Software Testing*.

2. Ejercicio 2

2.1. Diferencia entre *fault* y *failure*

Aunque suelen usarse como sinónimos en el lenguaje informal, la teoría del testing los distingue con cuidado:

- **Fault (defecto):** es un problema estático que vive en el software, ya sea en el código, en el diseño o en la especificación. Existe se ejecute o no el programa.
- **Failure (falla):** es un comportamiento externo incorrecto, observable durante la ejecución, frente al comportamiento esperado.

Entre uno y otro aparece un tercer concepto, el **error**, entendido como un estado interno incorrecto que surge cuando el defecto efectivamente se ejecuta. La cadena causal habitual se resume así:

$$fault \rightarrow error \rightarrow failure$$

La consecuencia práctica es importante: un defecto puede convivir con un programa durante mucho tiempo sin manifestarse, porque para que la falla ocurra el estado erróneo tiene que llegar hasta una salida observable. Diseñar tests significa, en el fondo, construir entradas que recorran ese camino completo.

3. Ejercicio 3

El enunciado presenta cuatro programas defectuosos junto con un caso de test que produce falla en cada uno. Para cada programa identificamos el defecto, proponemos una reparación y construimos los casos pedidos en los incisos b), c) y d). El criterio para esos casos sigue directamente la cadena RIPR: en (b) buscamos una entrada que ni siquiera ejecute la línea defectuosa; en (c), una entrada que la ejecute pero cuyo resultado siga coincidiendo con el esperado; y en (d), una entrada en la que el estado erróneo se propague hasta producir una falla observable.

3.1. Programa 1: `findLast(int[] x, int y)`

a) Defecto y corrección

El defecto está en la condición del ciclo:

```
1 for (int i = x.length - 1; i > 0; i--)
```

Como la guarda es `i > 0`, el índice 0 nunca llega a inspeccionarse y un valor ubicado al inicio del arreglo queda fuera del recorrido. La corrección consiste en relajar la condición:

```
1 for (int i = x.length - 1; i >= 0; i--)
```

b) Caso que no ejecute el defecto

No es posible con entradas no nulas: la condición defectuosa pertenece al `for` principal, por lo que toda invocación que recorra el arreglo la atraviesa.

c) Caso que ejecute el defecto y no produzca falla

`x = [5,2,3]`, `y = 2` Resultado esperado: 1. El 2 aparece en una posición distinta de 0, por lo que la omisión del primer índice no altera el resultado.

d) Caso que ejecute el defecto y produzca falla

`x = [2,3,5]`, `y = 2` Esperado: 0. Resultado defectuoso: -1, ya que el único 2 se encuentra precisamente en la posición que el ciclo nunca visita.

3.2. Programa 2: `lastZero(int[] x)`

a) Defecto y corrección

El método debe devolver el *último* índice cuyo valor es cero, pero recorre de izquierda a derecha y retorna el primero que encuentra. La corrección natural es invertir el sentido del recorrido:

```

1 for (int i = x.length - 1; i >= 0; i--) {
2     if (x[i] == 0) {
3         return i;
4     }
5 }

```

b) Caso que no ejecute el defecto

No es posible con arreglos no nulos: la estrategia incorrecta de recorrido constituye el cuerpo principal del método.

c) Caso que ejecute el defecto y no produzca falla

$x = [1, 0, 2]$ Resultado esperado: 1. Como hay un único cero, el primero y el último coinciden.

d) Caso que ejecute el defecto y produzca falla

$x = [0, 1, 0]$ Esperado: 2. Resultado defectuoso: 0, porque la implementación se queda con el primer cero que aparece y nunca llega al final del arreglo.

3.3. Programa 3: `countPositive(int[] x)`

a) Defecto y corrección

La condición usada es $x[i] \geq 0$, lo que provoca que el cero también se contabilice como positivo. La condición correcta debe ser estricta:

```

1 if (x[i] > 0) {
2     count++;
3 }

```

b) Caso que no ejecute el defecto

$x = []$. Con un arreglo vacío el cuerpo del `for` no se ejecuta y la comparación defectuosa nunca se evalúa.

c) Caso que ejecute el defecto y no produzca falla

$x = [-4, 2, 2]$ Resultado esperado: 2. La condición se evalúa para cada elemento, pero al no haber ningún cero el resultado coincide con el correcto.

d) Caso que ejecute el defecto y produzca falla

`x = [-4,2,0,2]` Esperado: 2. Resultado defectuoso: 3, ya que el cero se cuenta indebidamente.

3.4. Programa 4: `oddOrPos(int[] x)`

a) Defecto y corrección

La implementación usa `x[i] % 2 == 1` para detectar impares. El problema es que en Java el operador `%` preserva el signo del dividendo, así que `-3 % 2` vale `-1` y los impares negativos se pierden. La condición correcta verifica explícitamente que el resto sea distinto de cero:

```
1 if (x[i] % 2 != 0 || x[i] > 0) {  
2     count++;  
3 }
```

b) Caso que no ejecute el defecto

`x = []`. Sin elementos, la condición lógica no se evalúa.

c) Caso que ejecute el defecto y no produzca falla

`x = [2,4,-2]` Resultado esperado: 2. Aunque la condición defectuosa se evalúa en cada iteración, ningún elemento es un impar negativo, así que el resultado no se ve afectado.

d) Caso que ejecute el defecto y produzca falla

`x = [-3,-2,0,1,4]` Esperado: 3. Resultado defectuoso: 2: el `-3` debería contarse como impar y queda fuera.

3.5. Síntesis del ejercicio

Los cuatro defectos comparten un patrón habitual: errores de borde u omisiones en la condición lógica que rige al ciclo o al filtro. En particular,

- `findLast` no evalúa el índice 0;
- `lastZero` devuelve el primer cero en lugar del último;
- `countPositive` contabiliza el 0 como positivo;
- `oddOrPos` no reconoce impares negativos por la semántica de `%` en Java.

4. Ejercicio 4

A partir del análisis del ejercicio anterior se implementaron las cuatro reparaciones y se trasladaron los casos de prueba a JUnit, de modo de verificar empíricamente que las correcciones efectivamente eliminan las fallas observadas.

4.1. Reparaciones implementadas

Cada reparación es mínima e interviene únicamente sobre la condición que provocaba el comportamiento incorrecto:

- `findLast`: la guarda del ciclo pasa de `i > 0` a `i >= 0`, incluyendo el índice 0 en el recorrido.
- `lastZero`: se invierte el sentido del recorrido para que el primer cero hallado sea, esta vez, el último del arreglo.
- `countPositive`: se reemplaza `>= 0` por `> 0`, dejando al cero fuera del conteo.
- `oddOrPos`: se sustituye `% 2 == 1` por `% 2 != 0`, captando también a los impares negativos.

4.2. Casos de prueba JUnit

Se reutilizaron los casos del enunciado, dado que justamente eran los que evidenciaban las fallas en las versiones defectuosas:

- `findLast([2,3,5], 2) = 0`
- `lastZero([0,1,0]) = 2`
- `countPositive([-4,2,0,2]) = 2`
- `oddOrPos([-3,-2,0,1,4]) = 3`

4.3. Detección de falla usando el modelo RIPR

El modelo RIPR (*Reachability, Infection, Propagation, Reveability*) describe las cuatro condiciones que debe satisfacer un test para revelar un defecto: alcanzarlo, infectar el estado, propagar ese estado erróneo hasta una salida observable y, por último, exponerlo a través de una aserción. Para cada uno de los programas analizados, los casos de prueba elegidos las cumplen como sigue.

1) findLast

- **Reachability:** el test $(\{2,3,5\}, 2)$ entra al ciclo, que es donde vive el defecto.
- **Infection:** la guarda $i > 0$ omite la iteración correspondiente al índice 0, justamente la posición donde está el valor buscado.
- **Propagation:** al no encontrar coincidencia, el método retorna -1 en lugar de 0; el estado erróneo llega íntegro a la salida.
- **Revealability:** la aserción `assertEquals(0, ...)` compara con el valor correcto y deja en evidencia la divergencia.

2) lastZero

- **Reachability:** con $x = [0,1,0]$ el bucle se ejecuta y la condición se evalúa sobre cada posición.
- **Infection:** al recorrer de izquierda a derecha, el primer cero seleccionado es el de la posición 0, no el último.
- **Propagation:** ese índice se retorna inmediatamente, sin oportunidad de corrección posterior.
- **Revealability:** la aserción detecta 0 cuando se esperaba 2.

3) countPositive

- **Reachability:** cada elemento de $[-4,2,0,2]$ pasa por la comparación.
- **Infection:** la condición $x[i] \geq 0$ hace que el cero contribuya al contador, contaminando el estado interno.
- **Propagation:** ese incremento espurio se acumula en la variable que finalmente se retorna.
- **Revealability:** la aserción muestra un total de 3 en lugar de 2.

4) oddOrPos

- **Reachability:** con $x = [-3,-2,0,1,4]$ la condición lógica se evalúa para todos los elementos.
- **Infection:** $-3 \% 2$ vale -1, no 1, y por eso el impar negativo queda fuera del conteo.
- **Propagation:** el contador acumula una unidad menos de las debidas.

- **Revealability:** la aserción detecta 2 en lugar de 3.

4.4. Ejecución de pruebas

La suite se ejecuta desde la carpeta del ejercicio con:

```
mvn -Dmaven.repo.local=.m2 test
```

5. Ejercicio 5

Se analizan las clases `Point` y `ColorPoint` del paquete `practico1_exercises.color_points`. Se trata del ejemplo clásico de Bloch (*Effective Java*, capítulo 3) sobre el contrato de `equals` en jerarquías por herencia: cuando una subclase agrega estado de valor, mantener simultáneamente las propiedades de reflexividad, simetría y transitividad de `equals` se vuelve, en general, imposible.

5.1. Análisis de Point

a) Problema y modificación propuesta

`Point` define la igualdad por valor y, mediante `instanceof Point`, acepta como contrapartes válidas también a las instancias de cualquier subclase. Eso introduce una asimetría latente: una `ColorPoint` con las mismas coordenadas que un `Point` será considerada igual desde el lado de `Point`, aunque su `equals` sea más estricto y no acepte la comparación inversa.

La salida recomendada por Bloch es no extender `Point` para agregar estado de valor. En su lugar conviene usar composición — mantener un `Point` interno — o, si se prefiere prevenir la herencia, declarar `Point` como `final`.

b) Test que no ejecute la falla

La comparación entre dos `Point` con las mismas coordenadas no involucra subclases, por lo que el escenario problemático ni siquiera se ejecuta.

c) Test que ejecute la falla sin error observable

Comparación de `Point(1,2)` con `ColorPoint(2,3,RED)`. La rama conflictiva (un `Point` aceptando una subclase como contraparte) se recorre, pero como las coordenadas difieren el resultado coincide con el esperado (`false`).

d) Test con error pero sin falla

No identificamos un caso de este tipo: dada la simplicidad de `equals`, en cuanto el estado interno se infecta el resultado retornado coincide directamente con la salida observable, por lo que el error y la falla se manifiestan en el mismo momento.

5.2. Análisis de `ColorPoint`

a) Problema y modificación propuesta

`ColorPoint` redefine `equals` exigiendo `instanceof ColorPoint`. Combinado con la implementación heredada de `Point`, eso rompe la simetría del contrato: para `p = Point(1,2)` y `cp = ColorPoint(1,2,RED)` se cumple

$$p.equals(cp) == \text{true} \quad \text{y} \quad cp.equals(p) == \text{false}.$$

La corrección utilizada es la versión por composición `ColorPointFixed`, que ya no extiende `Point` y por lo tanto no entra en conflicto con su `equals`.

b) Test que no ejecute la falla

Comparación entre dos `ColorPoint` con mismas coordenadas y mismo color: el problema sólo aparece al cruzar tipos.

c) Test que ejecute la falla sin error observable

Comparación `ColorPoint(1,2,RED)` con `Point(4,5)`. Se transita la rama crítica, pero como las coordenadas son distintas el resultado correcto sigue siendo `false`.

d) Test con error pero sin falla

No se identifica un caso, por la misma razón conceptual señalada para `Point`: el resultado de `equals` es a la vez el estado interno y la salida observable.

Ejercicio 6

6.1. Aclaración sobre el material provisto

El material no incluía la clase `PointTest` ni el paquete `practico1_exercises.point_set`. Para poder resolver el ejercicio se reconstruyeron los artefactos faltantes:

- una implementación propia de `Point`;
- una clase de pruebas `PointTest`;

- tests adicionales para cubrir el comportamiento de `Point` dentro de un `HashSet`.

6.2. a) Nuevos tests, falla detectada y corrección

Los casos diseñados apuntan a verificar tanto la semántica de igualdad por valor como su correcto funcionamiento en estructuras basadas en hashing:

- igualdad entre dos puntos con las mismas coordenadas;
- desigualdad cuando alguna coordenada cambia;
- ausencia de duplicados al insertar puntos equivalentes en un `HashSet`;
- reconocimiento de un punto equivalente mediante `contains`.

La falla detectada fue la inconsistencia entre `equals` y `hashCode`: dos puntos considerados iguales por `equals` podían producir códigos de hash distintos, lo que rompía la promesa de `HashSet` y `HashMap`. La reparación es la canónica:

```
1 @Override
2 public int hashCode() {
3     return Objects.hash(x, y);
4 }
```

Con `hashCode` consistente, los tests sobre `HashSet` pasan a comportarse como se espera.

6.3. b) Identificación de Arrange–Act–Assert

Cada test se redactó dejando explícitas las tres fases del patrón AAA:

- **Arrange:** preparación del escenario y los datos;
- **Act:** ejecución de la operación bajo prueba;
- **Assert:** verificación del resultado esperado.

La división se mantiene también con comentarios en el código para que la intención de cada bloque quede a la vista.

6.4. Ejecución de pruebas

```
mvn -Dmaven.repo.local=.m2 test
```

7. Ejercicio 7

7.1. Aclaración sobre el material provisto

Tampoco se contaba con una implementación dada de `PointSet` ni con la clase de tests asociada. Como continuación natural del ejercicio anterior, se construyeron ambas y se incorporaron pruebas adicionales centradas en la semántica del conjunto.

7.2. Tests implementados

Los casos cubren las situaciones más representativas del comportamiento de un conjunto de puntos:

- estado inicial vacío;
- no inserción de duplicados cuando se agregan puntos equivalentes;
- reconocimiento de equivalencia en `contains`;
- eliminación de un punto a partir de otra instancia equivalente con `remove`;
- respuesta negativa de `contains` cuando el punto no fue agregado.

7.3. Resultado y análisis

La suite pasa sin fallas sobre la implementación actual de `PointSet`, que se apoya internamente en un `HashSet<Point>`. Este resultado es coherente con la corrección aplicada en el ejercicio anterior: una vez que `equals` y `hashCode` de `Point` son consistentes entre sí, las operaciones del `HashSet` se comportan correctamente y `PointSet` hereda ese buen comportamiento sin necesidad de ajustes propios.

7.4. Identificación de Arrange–Act–Assert

Como en el ejercicio anterior, todos los tests se estructuraron con las tres fases explícitamente delimitadas, manteniendo el mismo estilo de comentarios.

Conclusiones

El Práctico 1 funcionó como una primera puesta en práctica del vocabulario y las herramientas que se siguen usando a lo largo del cuatrimestre. En particular, permitió:

- distinguir con precisión los conceptos de *fault*, *error* y *failure*, y entender por qué un defecto puede pasar desapercibido aunque el código se ejecute con frecuencia;
- analizar y reparar defectos concretos en programas imperativos pequeños, prestando atención a sus condiciones de borde;
- aplicar el modelo RIPR para argumentar por qué cada test efectivamente detecta o no un defecto;
- revisar un problema clásico de diseño orientado a objetos — la dificultad de mantener simétrica y transitiva la igualdad de `Point` y `ColorPoint` — y resolverlo mediante composición;
- afianzar el diseño de pruebas JUnit con el patrón *arrange-act-assert* y la consistencia entre `equals` y `hashCode` en estructuras basadas en hashing.